

Recursividad: ejercicio práctico

Juan Jose Duarte Urrego

Estructuras de datos

Diana Lucia Rios Rojas

Institucion Universitaria Pascual Bravo

9/02/2026

1. tabla

Buena práctica	Código donde se genera la buena práctica
Caso base y reducción	if exp == 0: return 1 (Caso base) y el uso de exp - 1 (Reducción) dentro de la llamada.
Hipótesis inductiva (lo que provoca la recursión)	La instrucción base * potencia(base, exp - 1). Aquí se asume que la función ya sabe resolver el problema para un exponente más pequeño.
Sin sobrecarga innecesaria (cantidad de procesos)	El uso de una recursión lineal simple. Solo se realiza una multiplicación y una llamada recursiva por nivel, evitando cálculos duplicados.
Profundidad controlada (total de llamadas)	El control sobre el parámetro exp. Al restar 1 en cada paso hacia el cero, garantizamos que el total de llamadas sea exactamente exp + 1.

2. ejercicio

```
def potencia(base, exp):
```

```
    # Caso base: cualquier número elevado a 0 es 1
```

```
    if exp == 0:
```

```
return 1
```

```
# Caso recursivo: base * base^(exp-1)
```

```
else:
```

```
    return base * potencia(base, exp - 1)
```

```
# Ejemplo de uso: 2 elevado a la 3
```

```
resultado = potencia(2, 3)
```

```
print(f"El resultado es: {resultado}")
```

3. FOTOS

The screenshot displays a Python 3.11 IDE interface. On the left, a code editor shows a recursive function `potencia` that calculates the power of a base to an exponent. The function has a base case for `exp == 0` returning 1, and a recursive case for `exp > 0` returning `base * potencia(base, exp - 1)`. Below the code editor, a progress bar indicates the execution is at Step 15 of 19. On the right, a 'Print output' area is empty. Below it, a 'Frames' and 'Objects' panel shows the current state of the program. The 'Frames' panel lists the 'Global frame' and three instances of the 'potencia' function. The 'Objects' panel shows the function object for 'potencia' and the arguments 'base' and 'exp' for each instance. The bottom instance of 'potencia' shows 'base' as 2, 'exp' as 0, and a 'Return value' of 1.

```
Python 3.11
known limitations

1 def potencia(base, exp):
2     # Caso base: cualquier número elevado a 0 es 1
3     if exp == 0:
4         return 1
5     # Caso recursivo: base * base^(exp-1)
6     else:
7         return base * potencia(base, exp - 1)
8
9 # Ejemplo de uso: 2 elevado a la 3
10 resultado = potencia(2, 3)
11 print(f"El resultado es: {resultado}")

Edit this code

line that just executed
next line to execute

Step 15 of 19

NEW: teachers get free access to ad-free/AI-free mode

Move and hide objects (instructions)
```

Print output (drag lower right corner to resize)

Frames

Global frame

potencia

potencia

base 2

exp 3

potencia

base 2

exp 2

potencia

base 2

exp 1

potencia

base 2

exp 0

Return value 1

Python 3.11
[known limitations](#)

```
1 def potencia(base, exp):
2     # Caso base: cualquier número elevado a 0 es 1
3     if exp == 0:
4         return 1
5     # Caso recursivo: base * base^(exp-1)
6     else:
7         return base * potencia(base, exp - 1)
8
9 # Ejemplo de uso: 2 elevado a la 3
10 resultado = potencia(2, 3)
11 print(f"El resultado es: {resultado}")
```

[Edit this code](#)

→ line that just executed
→ next line to execute

<< First < Prev Next > Last >>

Done running (19 steps)

Print output (drag lower right corner to resize)

El resultado es: 8

Frames

Objects

Global frame

potencia

resultado

8

function
potencia(base, exp)